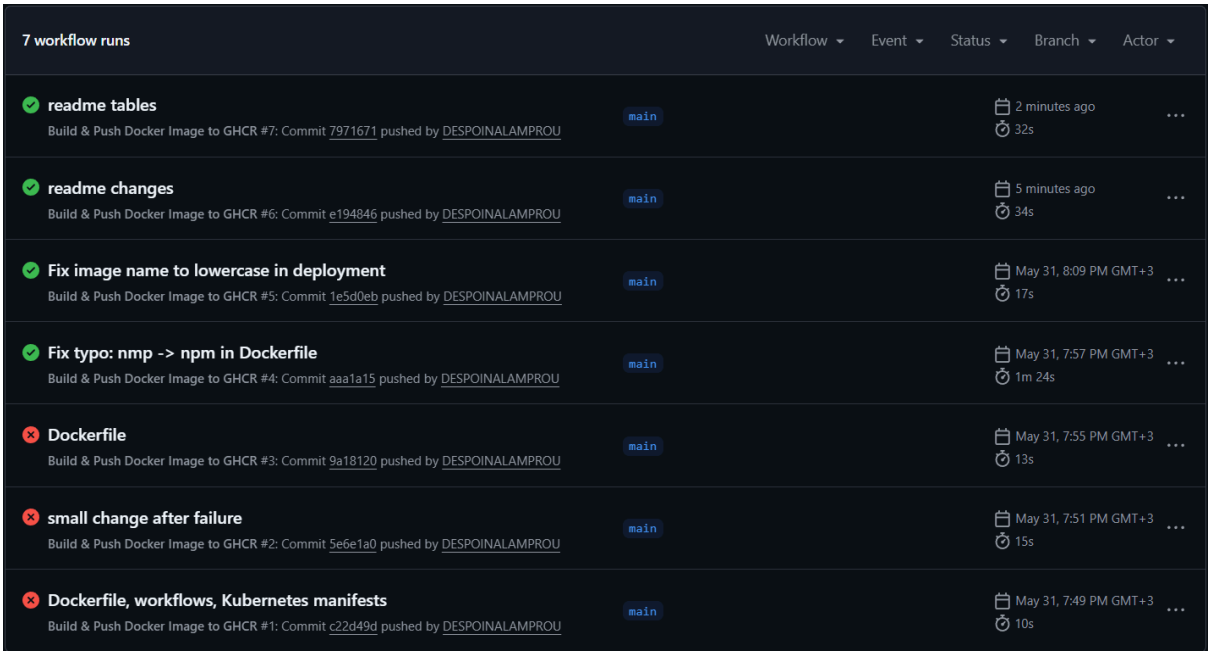
Software Engineering in Practice - Assignment 1

Δέσποινα Λάμπρου (t8230072)

GitHub Repository Link

https://github.com/DESPOINALAMPROU/seip_assignment_1_2026

CI/CD Proof



Cluster State Proof

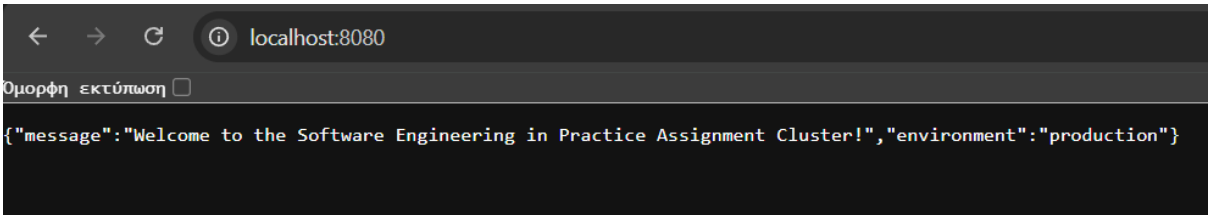
```
C:\Users\despi\seip_assignment_1_2026>kubectl get all -n default
NAME                                    READY   STATUS    RESTARTS   AGE
pod/echo-api-deployment-7c7f68cdc7-f8g6t   1/1     Running   0          21h
pod/echo-api-deployment-7c7f68cdc7-rggbt   1/1     Running   0          21h
pod/echo-api-deployment-7c7f68cdc7-w8mzt   1/1     Running   0          21h

NAME                       TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
service/echo-api-service   ClusterIP   10.96.135.34   <none>        80/TCP    22h
service/kubernetes         ClusterIP   10.96.0.1      <none>        443/TCP   8d

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/echo-api-deployment   3/3     3            3           22h

NAME                                          DESIRED   CURRENT   READY   AGE
replicaset.apps/echo-api-deployment-767b85bf6f   0         0         0       22h
replicaset.apps/echo-api-deployment-7c7f68cdc7   3         3         3       21h
```

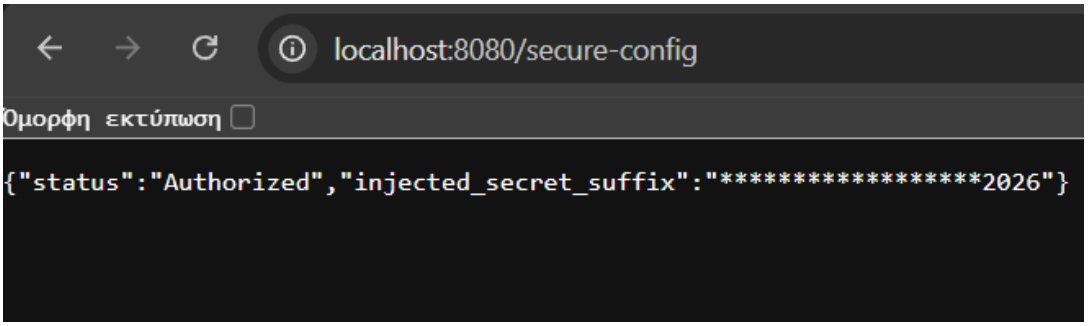
```
C:\Users\despi\seip_assignment_1_2026>kubectl get configmap,secret
NAME                           DATA   AGE
configmap/echo-api-config      2      22h
configmap/kube-root-ca.crt     1      8d

NAME                       TYPE     DATA   AGE
secret/echo-api-secret     Opaque   1      22h
```

Application Verification Proof

```
C:\Users\despi\seip_assignment_1_2026>kubectl get all -n default
NAME                                    READY   STATUS    RESTARTS   AGE
pod/echo-api-deployment-7c7f68cdc7-f8g6t   1/1     Running   0          21h
pod/echo-api-deployment-7c7f68cdc7-rggbt   1/1     Running   0          21h
pod/echo-api-deployment-7c7f68cdc7-w8mzt   1/1     Running   0          21h

NAME                       TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
service/echo-api-service   ClusterIP   10.96.135.34   <none>        80/TCP    22h
service/kubernetes         ClusterIP   10.96.0.1      <none>        443/TCP   8d

NAME                                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/echo-api-deployment   3/3     3            3           22h

NAME                                             DESIRED   CURRENT   READY   AGE
replicaset.apps/echo-api-deployment-767b85bf6f   0         0         0       22h
replicaset.apps/echo-api-deployment-7c7f68cdc7   3         3         3       21h
```

```
C:\Users\despi\seip_assignment_1_2026>kubectl get configmap,secret
NAME                           DATA   AGE
configmap/echo-api-config      2      22h
configmap/kube-root-ca.crt     1      8d

NAME                       TYPE     DATA   AGE
secret/echo-api-secret     Opaque   1      22h
```





AI Usage & Future Engineering Report

AI Integration

During this assignment, I used Claude as a supporting tool throughout the process. My approach was not to ask for ready-made solutions, but to use it as an interactive resource to understand concepts I had not encountered before, such as Docker layer caching, Kubernetes manifest structure, and Base64 secret encoding.

For each file I had to produce (Dockerfile, ci-cd.yaml, the k8s/ manifests), I first asked for a detailed explanation of the file's key elements and specifically what the file is supposed to do and why it is structured the way it is. Only then did I write or review the actual content. The goal was to understand what I was submitting, in which form and for what purpose, not just submit something that works.

Utility Analysis

The most valuable aspect of AI assistance was concept explanation from scratch. Coming into this assignment with no prior Docker or Kubernetes knowledge and experience, having something that could explain in simple terms and with ideal examples what a layer is, why the order of COPY and RUN instructions matters for caching, or what the difference between a liveness and readiness probe is, saved a significant amount of time compared to reading documentation alone.

The second most useful area was debugging. When errors appeared in the GitHub Actions pipeline, pasting the error message and getting a focused explanation of what actually went wrong made the troubleshooting process much more direct. Especially after having dedicated much time to write the files, detecting the slight, not easily noticeable errors could have been really exhausting and time-consuming.

Friction Points

AI assistance was not without its limitations. Several issues required manual intervention and independent troubleshooting:

Generally, I have to admit that the pipeline failed three times before succeeding.

- **CI/CD pipeline failures**: AI failed to catch a series of minor human errors during code reviews. In one instance, a missing destination argument in the Dockerfile (COPY . . written as COPY ..) caused a parse error. In another, a subtle typo (nmp install instead of npm install) passed through the AI's review undetected. I had to manually look through the raw workflow logs to diagnose and fix these syntax errors. The third was a permissions issue with GitHub Actions not being allowed to push packages to GHCR, which required manually changing the workflow permissions in the repository settings.
- **Lowercase image name**: Kubernetes rejected the image reference ghcr.io/DESPOINALAMPROU/echo-api:latest because container registry names

must be lowercase. This was not flagged in advance and only surfaced at runtime as an InvalidImageName error on the pods.

These were real friction points that required reading error messages carefully and making independent decisions, which was undoubtedly the most valuable part of the learning process. These friction points also demonstrated that while AI is a really helpful and useful tool, engineering success ultimately depends on an engineer's ability to carefully read error outputs, inspect cluster states, and make deliberate, independent fixes.

Future Architectural Outlook

If I were to extend this project toward a real-world production system, there are several next steps:

Replace port-forwarding with an Ingress Controller: The current setup uses kubectl port-forward to access the service locally, which is a development-only bypass. In production, an Ingress Controller (such as NGINX Ingress) would expose the application through a proper hostname with TLS termination, without requiring a manual tunnel from the developer's machine, increasing scalability.

Establish a GitOps workflow with ArgoCD: Right now, applying changes to the cluster requires running kubectl apply manually. With ArgoCD, the cluster would automatically sync itself to whatever is declared in the Git repository. In this way the repository becomes the single source of truth for the infrastructure state. Any change pushed to Git is automatically reflected in the cluster.

Add Prometheus monitoring: The application already has a /health endpoint, but there is no visibility into what happens over time. For instance how many requests come in, how long they take, whether pods are restarting. Adding Prometheus and Grafana would provide real-time metrics and dashboards, which are essential for operating any production system.

Integrate image scanning in the CI pipeline: Currently the pipeline builds and pushes the Docker image without checking it for known vulnerabilities. Adding a scanning step inside the GitHub Actions workflow would catch security issues in the base image or dependencies before the image ever reaches the registry.